

PROYECTO INTERMODULAR 2

SPRINT 2 – INFORME DE GESTION DE SCRUM

Miriam Mendieta Racedo.-
2DAMS
25/10/2026

SPRINT 2 – INFORME DE GESTIÓN DE SCRUM

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 25 de octubre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---------- **1**
EVOLUCIÓN DEL TRABAJO Y ESTADO DE HISTORIAS DE USUARIO (SPRINT 2) ---------- **4**
A. Historias de Usuario Completadas ---------- 4
B. Dificultades / Impedimentos Técnicos Solventados ---------- 14
C. Tareas No Finalizadas Traspasadas al Sprint 2 ---------- 15
D. Objetivo del Sprint 3 (Sprint Goal). ---------- 15
ANEXO ---------- **17**
BIBLIOGRAFÍA ---------- **21**

INTRODUCCIÓN

Al iniciar este proyecto, el mayor desafío técnico no fue la sintaxis del lenguaje, sino tener que **aprender Spring Boot al mismo tiempo que desarrollaba el primer sprint**. Cada error me obligaba a detenerme, investigar, probar, corregir… y volver a empezar. Esa combinación de descubrimiento y desarrollo me hizo llegar justísima a la entrega, con la sensación de haber sobrevivido más que de haber trabajado con seguridad.

Pero algo mucho más revelador ocurrió: mi cabeza hizo *click*. Pasé de pensar en **ejercicios aislados** a enfrentarme a un **sistema integral**, donde cada pieza es una ficha de dominó capaz de derribar a las demás.

En los ejercicios de cada lenguaje que aprendemos en la formación, el código vive en entornos controlados, con datos volátiles en memoria y sin dependencias externas. Sin embargo, esta app exige la convergencia de múltiples capas reales:

- **Persistencia:** Gestión del estado mediante una base de datos.
- **Presentación:** Renderizado dinámico del lado del servidor con Thymeleaf, HTML y CSS.
- **Control de Flujo:** Desacoplamiento de la lógica de negocio del protocolo HTTP.

Modificar una vista podía romper el sistema entero. Y la única forma de comprobar si todo seguía funcionando era abrir el navegador, rellenar formularios una y otra vez, esperando que nada hubiera caído por el camino. Era un flujo agotador, frágil y poco profesional.

Fue en ese caos donde entendí la verdad que marca este Sprint 2: si en cada paso **no refactorizo, el sistema se vuelve rígido y repetitivo; si no testeo, el sistema se vuelve impredecible.**

En síntesis, construir software no es solo escribir código, también es metodología, refactorización, testing y validación para evitar acoplamientos y prevenir fallos en cadena.

Estrategia de Mitigación: Refactoring y Testing Automatizado

A. *Arquitectura y Refactoring (Desacoplamiento)*

El primer paso consiste en aislar las responsabilidades aplicando el principio de responsabilidad única (SRP):

- ***Separación por Capas:*** Se dividió estrictamente el proyecto en "Controllers" (Thymeleaf/HTTP), "Services" (Lógica de negocio pura) y "Repositories" (Acceso a datos). Esto ya esta separado gracias a la implementación del proyecto a traves de Spring boot.

- ***Inyección de Dependencias:*** eliminación de instanciación directa de objetos (uso de "new") en favor de la inyección por constructor nativa de Spring Boot. Esto rompe el acoplamiento y permite la sustitución de componentes. Toca implementarlo a través de investigación y estudio del tema.

B. *Implementación del Testing Automatizado*

Con el código desacoplado, se implementa en una suite de pruebas automatizadas enfocada en la ***Pirámide de Pruebas*** , priorizando el retorno de inversión de tiempo y esfuerzo:

- ***Pruebas*** Unitarias Aisladas (JUnit 5 + Mockito): Se testea la lógica crítica (como el flujo de login o validaciones) de forma 100% aislada. Se utilizan "Mocks" para simular el comportamiento de la base de datos, garantizando que los tests se ejecuten sin depender de infraestructura externa.

- ***Flujo*** de Trabajo Paralelo: Cada funcionalidad nace acompañada de su test. No después, no "cuando haya tiempo". Justo después de implementarla. Así ninguna pieza queda expuesta y el sistema crece blindado desde dentro.

Con toda esta estrategia se pretende tener una mejora significativa en la gestión del tiempo y la calidad del código a presentar ya que combina una red de seguridad mediante tests, refactoring continuo para evitar deuda técnica y un desarrollo más ágil y eficiente que elimina el testing manual.

Reducción del Alcance del MVP

La decisión de disminuir nuevamente el alcance del MVP responde a una necesidad estratégica de garantizar la entrega de un producto **funcional, estable y demostrable** dentro del plazo límite del **22 de noviembre**. Durante el desarrollo inicial se identificó que el alcance previsto en la planificación original excedía la capacidad real de implementación dentro del tiempo disponible, especialmente considerando la complejidad de algunos módulos avanzados (gestión documental, alertas, OCR, backups, panel de activos completo, etc.).

Para evitar riesgos de retraso y asegurar un resultado optimo, se optó por **redefinir el MVP** centrándolo únicamente en los elementos esenciales que permiten validar el funcionamiento del sistema y demostrar su utilidad real:

- Gestión básica de activos digitales (CRUD).
- Registro de movimientos para trazabilidad.
- Roles y acceso diferenciado (usuario / administrador/Auditor).
- Gestión de usuario(Admistrador)
- Navegación mínima y estable.

Esta reducción garantiza que el producto entregado:

1. **Cumpla los requisitos mínimos funcionales** del proyecto.
2. **Sea demostrable** en vídeo sin errores críticos.
3. **Mantenga coherencia técnica** y una arquitectura escalable.
4. **Permita añadir módulos adicionales posteriormente** sin necesidad de rehacer la base del sistema.

Además, esta decisión se alinea con las buenas prácticas de desarrollo ágil: un MVP debe centrarse en las funcionalidades que aportan valor inmediato y permiten validar el concepto, dejando las mejoras y módulos complementarios para iteraciones posteriores.

En resumen, la reducción del alcance del MVP no implica una pérdida de calidad, sino una **optimización estratégica** para asegurar la entrega en fecha, evitar sobrecarga de trabajo y garantizar que el producto final sea **funcional, estable y ampliable**. Una vez entregado el MVP, se podrá continuará con el desarrollo progresivo de los módulos restantes, siguiendo una planificación más realista y sostenible.

Evolución del Trabajo y Estado de Historias de Usuario (Sprint 2)

A. Historias de Usuario Completadas

HU 01.1B – Acceso al Sistema: Logout

Descripción

Permite al usuario cerrar sesión de forma segura, invalidando la sesión activa y evitando el acceso a zonas privadas mediante navegación Atrás. Tras el logout, el sistema redirige al usuario a la pantalla de inicio de sesión.

Tareas técnicas

- Implementar endpoint de logout en Spring Security.
- Invalidar sesión correctamente.
- Configurar redirección al login tras cerrar sesión.
- Bloquear navegación atrás después del logout.
- Probar cierre de sesión desde distintas vistas.

Criterios de aceptación

1. **El usuario puede cerrar sesión correctamente**
   - Al hacer clic en “Cerrar sesión”, el sistema ejecuta el endpoint de logout configurado en Spring Security.
   - La sesión activa se invalida inmediatamente.
   - La autenticación del usuario se limpia por completo.
2. **Redirección obligatoria al login**
   - Tras el logout, el sistema redirige automáticamente a la pantalla de inicio de sesión.
   - No se muestra ninguna vista privada después del logout.

3. **Navegación atrás bloqueada**
   Si el usuario intenta volver atrás con el navegador después del logout:
   - No puede acceder a vistas privadas.
   - El sistema redirige a /login.
   - Las páginas privadas no se muestran desde caché.

4. **Acceso a zonas privadas prohibido**
   Cualquier intento de acceder a rutas privadas sin sesión activa:
   - Debe ser bloqueado por Spring Security.
   - Debe redirigir a /login.

5. **Pruebas funcionales**
   - Cerrar sesión desde cualquier vista privada funciona correctamente.
   - Intentar navegar atrás después del logout muestra login.
   - Intentar acceder manualmente a una URL privada después del logout muestra login.
   - La sesión no se mantiene en ningún caso tras el logout.

6. **Comportamiento consistente en todo el sistema**
   - El botón “Salir” que se utiliza para cerrar sesión está disponible en todas las vistas.
   - El comportamiento del logout es idéntico independientemente de la vista desde la que se ejecuta.

HU 03.2 – Visualización Segura de Contraseñas

Descripción

Mejora la usabilidad y seguridad de los formularios permitiendo mostrar u ocultar la contraseña mediante un botón de visualización, manteniendo el campo enmascarado por defecto.

Tareas técnicas

- Implementar campo de contraseña con enmascarado.
- Añadir botón “ver/ocultar contraseña”.
- Asegurar que el toggle no afecta la seguridad del formulario.
- Probar en login y recuperación de contraseña.

Criterios de aceptación

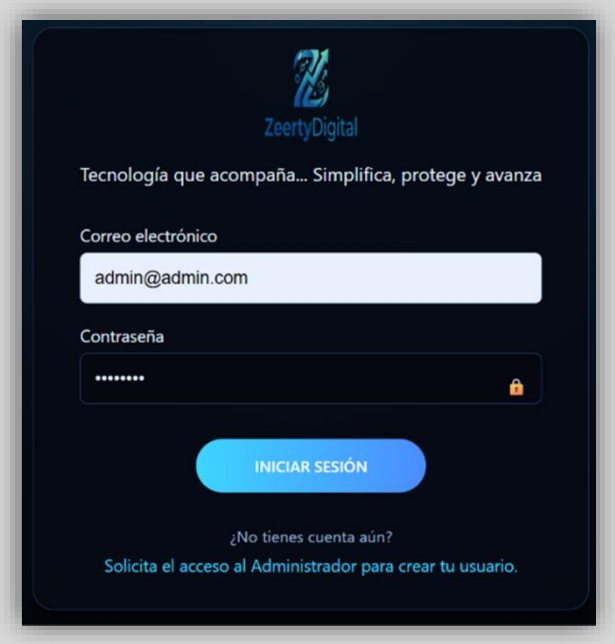
1. **El campo de contraseña está enmascarado por defecto**
    - El input debe mostrarse inicialmente como type="password".
    - El usuario no puede ver la contraseña hasta que pulse el botón de visualización.

2. **Existe un toggle para mostrar/ocultar la contraseña**
    - El toggle está claramente visible dentro del campo de contraseña.
    - El icono del toggle cambia según el estado: mostrar/ocultar.

3. **El toggle no compromete la seguridad**
    - El cambio de tipo del input **no debe afectar** la validación del formulario.
    - No debe permitir copiar la contraseña automáticamente.
    - No debe almacenar la contraseña en ningún lugar adicional.
    - No debe romper políticas de seguridad del sistema.

4. **No se permite dejar la contraseña visible tras enviar el formulario**
    - Al enviar el formulario, el campo debe volver a estar enmascarado.
    - No debe quedar visible en caso de error de validación.

5. **Pruebas funcionales**
    - Mostrar contraseña → funciona correctamente.
    - Ocultar contraseña → funciona correctamente.
    - El icono cambia según el estado.
    - El formulario sigue validando la contraseña igual que antes.
    - No se producen errores visuales ni de layout.





HU 01.X – Gestión de Usuarios por el Administrador

Descripción

El administrador puede dar de alta nuevos usuarios, editar sus roles y activar o desactivar cuentas. Esta funcionalidad permite controlar quién accede al sistema y con qué permisos.

Tareas técnicas

- Crear formulario de alta de usuario.
- Validar correo duplicado.
- Implementar endpoint para crear usuarios.
- Añadir selector de roles.
- Implementar activación/desactivación de usuarios.
- Proteger vista solo para ROLE_ADMIN.
- Probar alta, edición y desactivación.

Criterios de Aceptación

1. **Acceso exclusivo para ROLE_ADMIN**
    - Solo usuarios con **ROLE_ADMIN** pueden acceder a la vista de gestión de usuarios.
    - No se permite acceder mediante URL directa sin ROLE_ADMIN.

2. **Alta de nuevos usuarios**
    - El administrador puede abrir el formulario de alta de usuario.
    - El formulario contiene campos obligatorios: nombre, correo, rol, estado.
    - Si el correo ya existe en el sistema →se muestra error y no se crea el usuario.
    - Si los datos son válidos → el usuario se crea correctamente y aparece en la lista.

3. **Selector de roles funcional**
    - El administrador puede asignar un rol al usuario desde un selector (dropdown).
    - Los roles disponibles se muestran correctamente.
    - El rol seleccionado se guarda y afecta a los permisos del usuario.

4. **Edición de usuarios**
    - El administrador puede modificar datos básicos del usuario (nombre, rol, estado).
    - Si se intenta asignar un correo duplicado → se bloquea la edición y se muestra error.
    - Los cambios válidos se guardan correctamente y se reflejan en la lista.

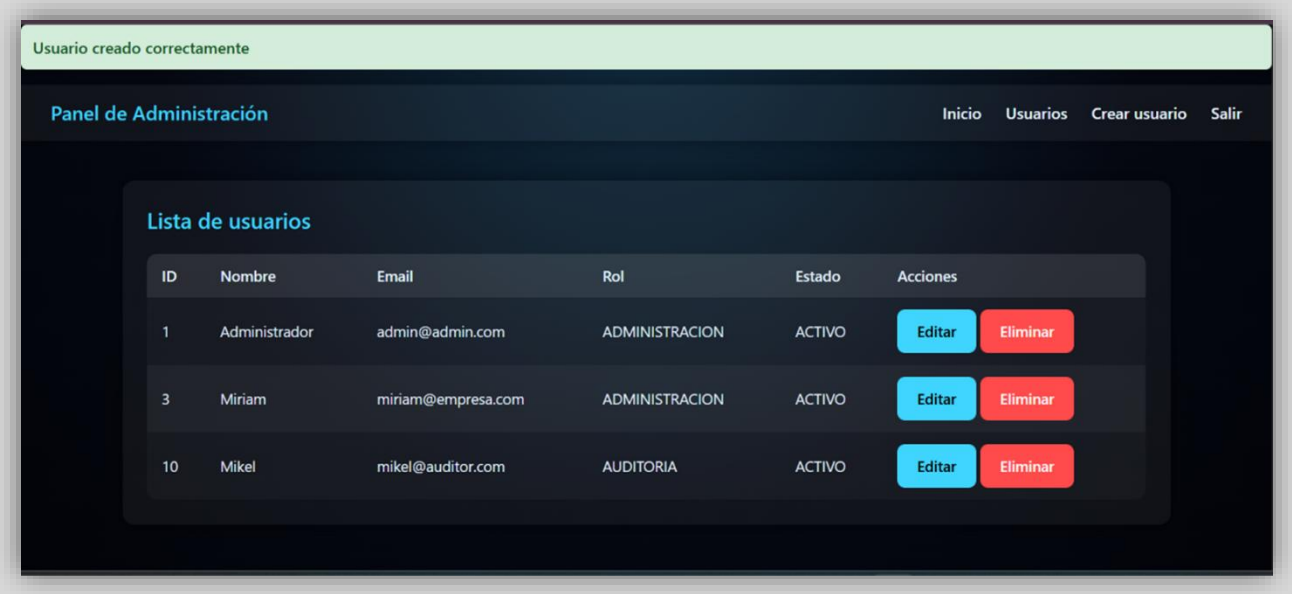
5. **Activación y desactivación de usuarios**
    - El administrador puede activar o desactivar usuarios desde la vista de gestión.
    - Un usuario desactivado **no puede iniciar sesión**.
    - El estado del usuario se actualiza correctamente en la base de datos.
    - La interfaz refleja el estado actualizado (activo/inactivo).

6. **Seguridad del módulo**
    - Todas las operaciones (alta, edición, activación/desactivación) están protegidas por Spring Security.
    - No se permite realizar acciones administrativas mediante llamadas directas sin ROLE_ADMIN.
    - No se filtra información sensible en errores o respuestas.

7. **Pruebas**
    - Alta de usuario con datos válidos → OK.
    - Alta de usuario con correo duplicado → ERROR.
    - Edición de usuario con datos válidos → OK.

- Edición con correo duplicado → ERROR.
- Activación/desactivación → OK.
- Intento de acceso sin ROLE_ADMIN → bloqueado.
- Validación de que un usuario desactivado no puede iniciar sesión. → esta prueba ha fallado se crea una nueva HU para poder solucionarla.

8. **Cumple la Definition of Done**
   - Código revisado.
   - Sin errores visibles.
   - Documentación mínima actualizada.
   - HU marcada como completada en Trello.



Nueva HU – Bloqueo de Inicio de Sesión para Usuarios Desactivados

Descripción

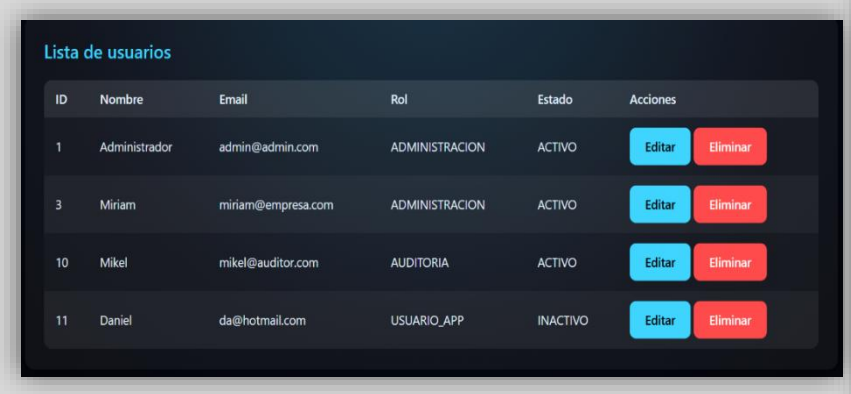
Cuando un usuario está desactivado, no debe poder iniciar sesión en el sistema. La autenticación debe comprobar el estado de la cuenta y bloquear el acceso si el usuario está inactivo, mostrando un mensaje claro.
Garantizar que solo usuarios activos puedan autenticarse, respetando la activación/desactivación gestionada por el administrador.

Tareas Técnicas

- Añadir o verificar el campo activo en la entidad Usuario Asegurar que el modelo tiene un booleano que representa si la cuenta está activa o desactivada.
- Actualizar el repositorio y DTOs si es necesario Confirmar que el estado activo se carga correctamente desde la base de datos.
- Implementar la validación del estado en UserDetails Sobrescribir isEnabled() para que devuelva el valor de usuario.activo.
- Añadir validación explícita en UserDetailsService Si el usuario está desactivado, lanzar DisabledException antes de devolver el User-Details.
- Configurar Spring Security para interpretar DisabledException Asegurar que el flujo de autenticación bloquea el login cuando isEnabled() es false.
- Implementar mensaje de error claro en el login Mostrar “Usuario desactivado” o mensaje equivalente cuando la autenticación falla por estado inactivo.
- Proteger endpoints privados contra usuarios desactivados Verificar que un usuario desactivado no puede acceder a rutas privadas aunque tenga credenciales correctas.
- Pruebas manuales: Desactivar usuario desde el panel de administración. Intentar iniciar sesión → debe fallar. Activarlo de nuevo →login debe funcionar.

Criterios de aceptación

- Un usuario desactivado no puede iniciar sesión.
- El sistema comprueba el estado activo/inactivo durante la autenticación.
- Si el usuario está desactivado, el login se bloquea y se muestra un mensaje de error claro (ej. “Usuario desactivado”).
- Un usuario activado puede iniciar sesión correctamente si las credenciales son válidas.
- La lógica de bloqueo está centralizada en el flujo de autenticación (UserDetails / UserDetailsService / filtro de seguridad).
- No se puede saltar la restricción mediante acceso directo a endpoints protegidos.
- Existen pruebas que validan: Login con usuario activo → OK. Login con usuario desactivado → ERROR.







B. Dificultades / Impedimentos Técnicos Solventados

Tipo de Impedimento	Descripción del Problema	Impacto en el Sprint	Estrategia de Solución	Medidas Preventivas / Lecciones Aprendidas
Reorganización del MVP	El MVP inicial no estaba bien definido y contenía funcionalidades que no eran prioritarias. Esto generó confusión sobre qué debía entregarse en cada sprint.	Retraso en tareas y necesidad de replantear el backlog.	Revisión completa del alcance, redefinición de módulos y ajuste del backlog para que cada sprint tuviera entregables concretos y alcanzables.	Validar el alcance del MVP antes de iniciar el desarrollo de acuerdo con la fecha de entrega final.
Validación Activo/inactivo	Spring Security, por defecto, solo valida usuario + contraseña.Por lo tanto un usuario desactivado puede iniciar sesión.	Riesgo de seguridad y comportamiento incorrecto del sistema.	Añadir validación manual en el servicio de login para comprobar el estado del usuario antes de permitir el acceso.	Revisar las configuraciones por defecto del framework; documentar reglas de negocio críticas.
Recordar hacer los commit	Se me olvida hacer commits importantes, dificultando el seguimiento del progreso.	Riesgo de pérdida de trabajo y problemas de sincronización.	Establecer una rutina de commits frecuentes.	Aplicar la regla “commit pequeño, commit seguro”.
Navegación del administrador	El administrador podía entrar en la vista de usuario (/home) pero no volver a su panel (/admin/home).	Navegación incompleta y mala experiencia de usuario.	Añadir un botón condicional en home.html usando el rol del usuario.	Probar rutas completas; validar roles en vistas y controladores.

c. Tareas No Finalizadas Traspasadas al Sprint 2

D. Objetivo del Sprint 3 (Sprint Goal).

Historias de Usuario seleccionadas del Product Backlog para este Sprint (SprintBacklog)

ID HU	Nombre	Descripción breve	Estimación (puntos)

Desglose inicial de tareas técnicas

Resumen del Sprint Goal

ANEXO

Seguridad en Spring Boot y Gestión de Usuarios Activos/Inactivos

La seguridad es uno de los pilares fundamentales en el desarrollo de aplicaciones empresariales. En cualquier sistema donde existan usuarios, roles, datos sensibles o procesos internos, es imprescindible garantizar:

- que solo usuarios autorizados accedan al sistema,
- que cada usuario tenga acceso únicamente a los recursos permitidos,
- que la autenticación sea robusta,
- que los datos estén protegidos frente a ataques externos,
- que el sistema pueda gestionar estados como cuentas bloqueadas, inactivas o caducadas.

Spring Security que se encuentra dentro del ecosistema Java y es fundamental para implementar estas medidas de seguridad ya que su diseño modular, flexible y extensible permite adaptarlo a cualquier tipo de aplicación, desde sistemas monolíticos hasta arquitecturas distribuidas.

Spring Security nació como Acegi Security en 2003, un framework independiente que ofrecía autenticación y autorización para aplicaciones Java. Debido a su éxito y adopción masiva, fue integrado oficialmente en Spring Framework bajo el nombre de Spring Security.

Arquitectura interna de Spring Security

Spring Security funciona mediante una arquitectura basada en filtros. Cada petición HTTP pasa por una cadena de filtros antes de llegar al controlador correspondiente.

- Security Filter Chain

  La cadena de filtros es el núcleo del sistema. Incluye filtros como:

  - Filtro de autenticación
  - Filtro de autorización
  - Filtro de manejo de excepciones
  - Filtro de sesión
  - Filtro de CSRF
  - Filtro de logout

  Cada filtro realiza una tarea específica y se ejecuta en un orden predefinido. Esto permite interceptar peticiones, validar credenciales, comprobar permisos y bloquear accesos no autorizados.

- AuthenticationManager → Es el componente que coordina el proceso de autenticación. Recibe las credenciales del usuario y decide si son válidas.

- AuthenticationProvider → Es el encargado de realizar la autenticación real. Spring Security incluye uno por defecto para autenticación basada en formularios, pero se pueden crear proveedores personalizados.

❖ UserDetails y UserDetailsService → Son dos interfaces fundamentales:

- UserDetails representa al usuario autenticado.
- UserDetailsService carga el usuario desde la base de datos.

En esta app son esenciales por ejemplo para implementar la HU de usuarios activos/inactivos.

Flujo completo de autenticación en Spring Security

El proceso de autenticación sigue estos pasos:

1. El usuario introduce sus credenciales en el formulario de login.
2. El formulario envía los datos al filtro de autenticación.
3. El filtro llama al UserDetailsService.
4. UserDetailsService busca el usuario en la base de datos.
5. Se crea un objeto UserDetails.
6. Spring Security valida:
   - ✓ contraseña,
   - ✓ roles,
   - ✓ estado de la cuenta,
   - ✓ si la cuenta está bloqueada,
   - ✓ si las credenciales han caducado.
7. Si todo es correcto, el usuario inicia sesión.
8. Si algo falla, se lanza una excepción y se bloquea el acceso.

Este flujo es totalmente configurable.

Implementación de CustomUserDetailsService

Esta clase se encarga de:

- buscar el usuario por email,
- verificar si existe,
- comprobar si está activo,
- devolver un objeto UserDetails.

Integración con SecurityConfig

La clase SecurityConfig permite personalizar:

- la página de login,
- el comportamiento en caso de error,
- la redirección cuando el usuario está inactivo,
- la protección de rutas privadas.

Mediante un AuthenticationFailureHandler, es posible detectar la excepción DisabledException y mostrar un mensaje específico.

Buenas prácticas de seguridad en Spring Boot

1. Usar contraseñas encriptadas con BCrypt.
2. No almacenar contraseñas en texto plano.
3. Separar la lógica de seguridad en paquetes específicos.
4. No exponer endpoints sin protección.
5. Usar HTTPS en producción.
6. Validar siempre el estado del usuario.
7. Registrar intentos fallidos de login.
8. Implementar bloqueo por intentos excesivos si es necesario.
9. Mantener actualizadas las dependencias de Spring Security.
10. Revisar periódicamente la configuración de roles y permisos.

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.
- Documentación oficial de Spring Boot Security: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#web.security